

PZ-PAJ7620 手势识别模块开发手册

本开发手册将教大家如何在普中 F407 最小系统&天马&麒麟开发板上使用 PZ-PAJ7620 手势识别传感器模块。本章分为如下几部分内容：

- 1 PZ-PAJ7620 模块简介
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-PAJ7620 模块简介

1.1 特性参数

PZ-PAJ7620 是深圳普中科技推出的一款高性能手势识别传感器模块。该模块采用原相科技（Pixart）公司的 PAJ7620U2 芯片，芯片内部集成了光学数组式传感器，以使复杂的手势和光标模式输出，自带九种手势识别，支持上、下、左、右、前、后、顺时针旋转、逆时针旋转和挥动的手势动作识别，以及支持物体接近检测等功能。

PZ-PAJ7620 模块具有：体积小、灵敏度高、支持中断输出、兼容 3.3V/5V 系统、使用方便等特点。

PZ-PAJ7620 模块各项参数如下图所示。

项目	说明
接口特性	3.3V/5V
通讯接口	IIC 接口
通讯速率	400Khz (Max)
工作环境光	<100K Lux
有效探测距离	5~15cm
手势识别种类	9 种（上、下、左、右、前、后、顺时针、逆时针、挥动）
工作温度	-20℃-70℃
存储温度	-40℃-85℃
模块尺寸	11.42mm*19.52mm
电源电压	3.3V/5V
IO 口电平 ^①	3.3V LVTTTL
功耗	3~10ma

注①：模块 IO 电压是 3.3V，不过我们做了 3.3/5V 兼容性处理（串 120R 电阻）所以可以直接连接 3.3V 和 5V 的 MCU 使用。

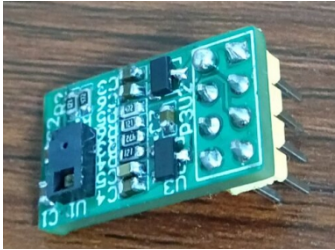
1.2 使用说明

1.2.1 模块引脚说明

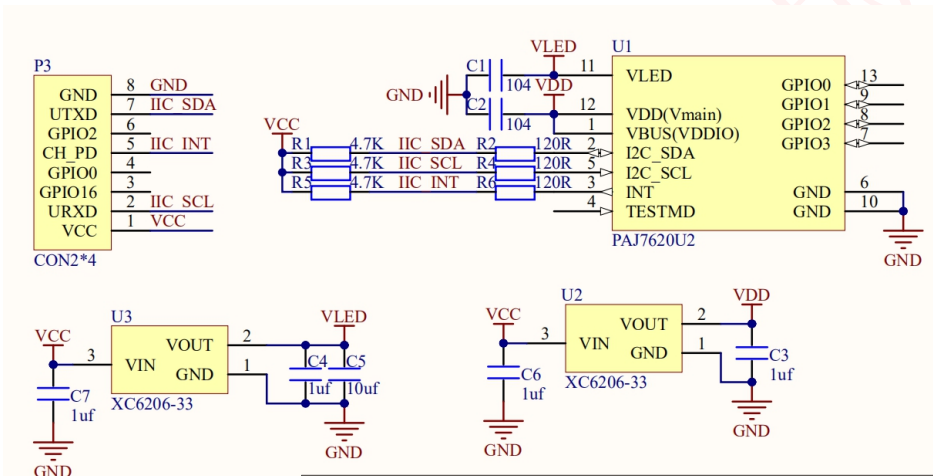
PZ-PAJ7620 手势识别传感器模块通过 2*4 的排针（2.54mm 间距）同外部连接，模块可以与天马&麒麟开发板直接对接（插 WIFI 接口），而 F407 最小系

统开发板则可以通过杜邦线连接模块进行测试。所有普中 STM32F407 系列开发板都可使用该例程，用户可以直接在这些开发板上，对模块进行测试。

PZ-PAJ7620 手势识别传感器模块外观如下图所示：



PZ-PAJ7620 模块原理图如下图所示：



从图中可以看出，模块自带了两个 3.3V 超低压差稳压芯片，给 PAJ7620 芯片供电，因此外部供电为 5V。模块通过 P3 排针与外部连接，引出了 VCC、GND、IIC_SDA、IIC_SCL、IIC_INT 信号，其中，IIC_SCL、IIC_SDA、IIC_INT 带了 4.7K 上拉电阻，外部可以不用再加上拉电阻了。

PZ-PAJ7620 手势识别传感器模块通过一个 2*4 的排针（P3）同外部电路连接，各引脚的详细描述如下图所示：

序号	名称	说明
1	VCC	3.3V/5V电源输入
2	IIC_SCL	IIC 通信时钟线
3	NC	
4	NC	
5	IIC_INT	中断输出引脚
6	NC	
7	IIC_SDA	IIC 通信数据线
8	GND	地线

模块通过 IIC 接口与外部进行通信，上电时，默认的 IIC 从机地址为：0X73

（使用时需左移一位）。

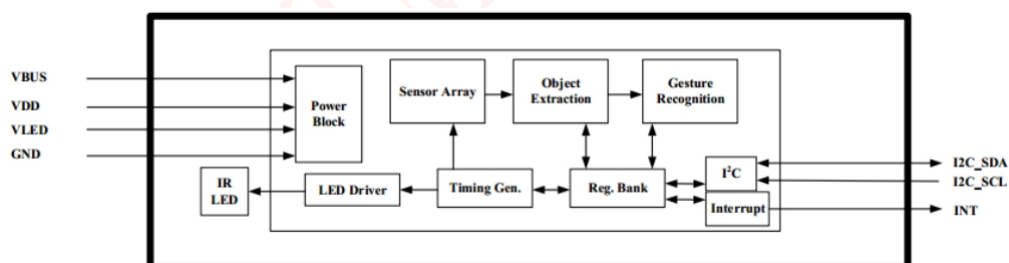
1.2.2 PAJ7620 简介

PAJ7620 是原相科技（PixArt）公司推出的一款光学数组式传感器，内置光源和环境光抑制滤波器集成的 LED，镜头和手势感测器在一个小的立方体模组，能在黑暗或低光环境下工作。同时传感器内置手势识别，支持 9 个手势类型和输出的手势中断和结果。并且内置还提供接近检测功能，可用于感测物体接近或离开。

PAJ7620 的特点包括：

- ①IIC 接口，支持高达 400Khz 通信速率。
- ②内置 9 个手势类型（上、下、左、右、前、后、顺时针旋转、逆时针旋转、挥动），支持输出中断。
- ③支持接近检测功能，检测物体体积大小和亮度。
- ④待机功耗电流 15uA。
- ⑤抗灯光干扰

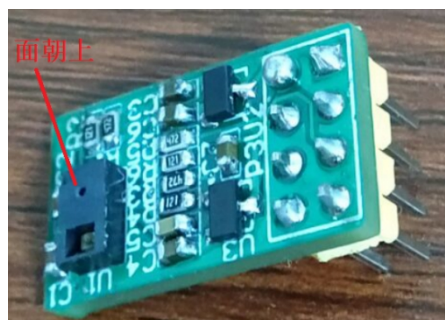
PAJ7620 的模块功能框图如下图所示：



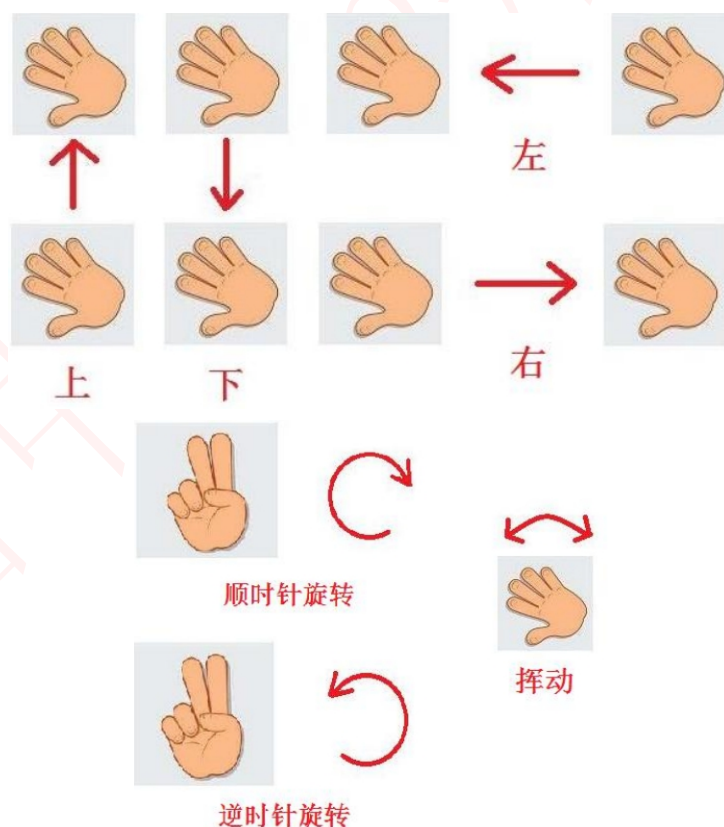
其中，I2C_SCL 和 I2C_SDA 是连接 MCU 的 IIC 接口，MCU 通过这个 IIC 接口来控制 PAJ7620，在图中可以看到，PAJ7620 内部自带 LED 驱动器，传感器感应阵列、目标信息提取阵列和手势识别阵列。PAJ7620 工作时通过内部 LED 驱动器，驱动红外 LED 向外发射红外线信号，当传感器阵列在有效的距离中探测到物体时，目标信息提取阵列会对探测目标进行特征原始数据的获取，获取的数据会存在寄存器中，同时手势识别阵列会对原始数据进行识别处理，最后将手势结果存到寄存器中，用户可根据 I2C 接口对原始数据和手势识别的结果进行读取。

1.2.3 手势操作说明

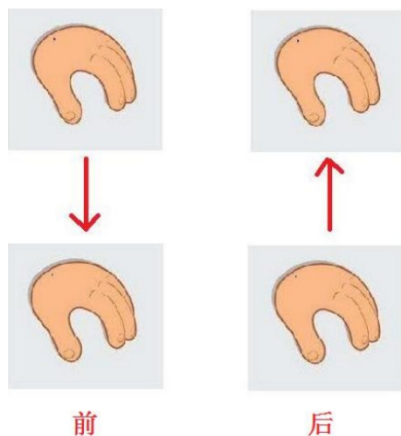
PAJ7620 内部自带了 9 个手势识别，分别是“上”、“下”、“左”、“右”、“前”、“后”、“顺时针旋转”、“逆时针旋转”、“挥动”。使用时传感器的开窗口位置需朝上，如下图所示：



手在传感器的上方，保持与传感器的垂直距离，做出下图所示的手势，可以分别得出“上”、“下”、“左”、“右”、“顺时针旋转”、“逆时针旋转”、“挥动”的识别结果。



手在传感器的上方，与传感器的垂直方向上距离有相对的变化，做出下图所示的手势，可以得出“前”、“后”的识别结果。



1.2.4 常用寄存器介绍

PAJ7620 内部总共有上百个寄存器，但官方的也只是对部分的寄存器进行讲解，这里我们仅介绍 PAJ7620 常用的几个寄存器。其他的请大家参考：PAJ7620 芯片手册。

这里我们先提及下，在 PAJ7620 的内部有两个 BANK 寄存器区域，分别是 BANK0 和 BANK1。不同的区域用于访问不同的功能寄存器，但想访问其中的 BANK 区域下的寄存器，需在访问前发送控制指令进入该寄存器区域，具体控制指令如下图所示：

BANK 区域	地址	数值
BANK0	0xEF	0x00
BANK1	0xEF	0x01

从图中可以看到，进入 BANK0 区域往传感器 0xEF 地址写 0x00 数值，而 BANK1 区域往传感器 0xEF 地址写 0x01 数值。

首先，我们介绍下 BANK1 下的 PAJ7620 使能工作寄存器，该寄存器地址为 0X72，寄存器描述如下图所示：

Register Bank 1, ADDR 0x72, Enable/Disable PAJ7620U2		
NAME	Reserved	Enable
BIT #	[7:1]	[0]
ACCESS	Write as 0	R/W
DEFAULT	0	0
	0x00	

NAME	FUNCTION/OPERATION
Enable	1: Enable PAJ7620U2 0: Disable PAJ7620U2

该寄存器用于使能 PAJ7620 工作，这里我们只关心 bit0 位，设置为 1，

则使能 PAJ7620 工作，设置为 0，则失能 PAJ7620 工作。

接下来我们介绍下 BANK0 下的挂起管理寄存器，该寄存器地址为 0X03，各位描述如下图所示：

Register Bank 0, ADDR 0x03, I ² C Suspend Command		
NAME	Reserved	Suspend
BIT #	[7:1]	[0]
ACCESS	Write as 0	W
DEFAULT	0	1
0x01		

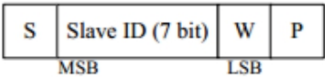
NAME	FUNCTION/OPERATION
Suspend	Write 1: Enter suspend state (wake up by writing I2C slave ID (default: 0x73), Refer to topic "I ² C Bus Timing Characteristics and Protocol")

其中，Suspend bit0 位用来控制挂起，要 PAJ7620 进入挂起状态，并非设置该位为 1 就可以，手册上有具体说明如何进入和退出挂起，如下图所示：

To enter the suspend state, first disable the PAJ7620U2 by writing Register Bank 1, ADDR 0x72 with 0x00 then process the I²C suspend command by writing Register Bank 0, ADDR 0x03 with 0x01.
To exit the suspend state, first process the I²C wake-up command by writing the slave ID (Refer to topic "I²C Bus Timing Characteristics and Protocol") then enable the PAJ7620U2 by writing Register Bank 1, ADDR 0x72 with 0x01.

进入挂起前，先配置 BANK1 的 PAJ7620 使能工作寄存器 0X72 为 0X00，失能 PAJ7620 工作，然后再往 BANK0 的挂起管理寄存器 0X03 写 0X01，才真正的挂起 PAJ7620。而退出挂起唤醒工作，则需执行 3 个步骤：

(1) 往 PAJ7620 发送写命令，以触发唤醒，命令格式如下图所示：



唤醒命令格式就是 I2C 通讯的命令格式，从图中可以看到，在发送写命令后不用等待 PAJ7620U2 的应答，直接发送停止就可以了。

(2) 发送完唤醒命令后，需等待大于 700us 的时间，然后读取 PAJ7620 的 0X00 寄存器，判断是否为 0X20 数值，若不是则继续执行步骤 1 继续唤醒。直到读取 0X00 寄存器值为 0X20，则唤醒成功。

(3) 唤醒成功后（由于在挂起时，把 PAJ7620 给关闭了）往 BANK1 的使能工作寄存器 0X72 写 0X01，使能 PAJ7620 工作。

按照上面的三个步骤，就可以让 PAJ7620 从挂起到唤醒工作了。这里需注意一下，PAJ7620 首次上电，传感器也是工作在挂起状态，同样也需唤醒其工作。

接着我们看下 BANK0 下的手势检测输出中断使能寄存器 1，该寄存器地址

为 0X41，各位描述如下图所示：

Register Bank 0, ADDR 0x41, Gesture Detection Interrupt Flag Mask								
NAME	Counter-Clockwise Mask	Clockwise Mask	Backward Mask	Forward Mask	Right Mask	Left Mask	Down Mask	Up Mask
BIT #	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
ACCESS	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
DEFAULT	1	1	1	1	1	1	1	1
0xFF								

NAME	FUNCTION/OPERATION
Counter Clockwise Mask	1: Counter clockwise gesture will generate an interrupt 0: Counter clockwise gesture will not generate an interrupt
Clockwise Mask	1: Clockwise gesture will generate an interrupt 0: Clockwise gesture will not generate an interrupt
Backward Mask	1: Backward gesture will generate an interrupt 0: Backward gesture will not generate an interrupt
Forward Mask	1: Forward gesture will generate an interrupt 0: Forward gesture will not generate an interrupt
Left Mask	1: Left gesture will generate an interrupt 0: Left gesture will not generate an interrupt
Right Mask	1: Right gesture will generate an interrupt 0: Right gesture will not generate an interrupt
Down Mask	1: Down gesture will generate an interrupt 0: Down gesture will not generate an interrupt
Up Mask	1: Up gesture will generate an interrupt 0: Up gesture will not generate an interrupt

该寄存器作用于手势识别，bit0~bit7 位用于使能不同手势识别结果的中断输出，默认值为 0XFF，其中 bit0 位为“上”、bit1 位为“下”、bit2 位为“左”、bit3 位为“右”、bit4 位为“前”、bit5 位为“后”、bit6 位为“顺时针旋转”、而 bit7 位为“逆时针旋转”。对应位设置为 1，则开启当检测到对应的手势识别时，会输出对应手势识别结果中断。若对应位设置为 0，则关闭手势识别结果中断。

接着我们看下 BANK0 下的手势检测输出中断使能寄存器 2，该寄存器地址为 0X42，各位描述如下图所示：

Register Bank 0, ADDR 0x42, Gesture Detection Interrupt Flag Mask		
NAME	Reserved	Wave Mask
BIT #	[7:1]	[0]
ACCESS	Write as 0000000	R/W
DEFAULT	1111111	1
0xFF		

NAME	FUNCTION/OPERATION
Wave Mask	1: Wave gesture will generate an interrupt 0: Wave gesture will not generate an interrupt

该寄存器也是作用于手势识别，其中只有 bit0 位有作用，bit1-bit7 为保留位，寄存器默认值为 0XFF。bit0 为用于使能手势识别“挥动”的输出中断，当 bit0 位设置为 1 时，则使能“挥动”手势识别输出中断，设置 0，则关闭输出中断。

接着我们看下 BANK0 的手势识别中断标志寄存器 1，该地址为 0X43，各位描述如下图所示：

Register Bank 0, ADDR 0x43, Gesture Detection Interrupt Flag								
NAME	Counter Clockwise	Clockwise	Backward	Forward	Right	Left	Down	Up
BIT #	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
ACCESS	R	R	R	R	R	R	R	R
DEFAULT	-	-	-	-	-	-	-	-

NAME	FUNCTION/OPERATION
Counter Clockwise	1: Counter clockwise gesture be detected 0: No Counter clockwise gesture be detected
Clockwise	1: Clockwise gesture be detected 0: No Clockwise gesture be detected
Backward	1: Backward gesture be detected 0: No Backward gesture be detected
Forward	1: Forward gesture be detected 0: No Forward gesture be detected
Right	1: Right gesture be detected 0: No Right gesture be detected
Left	1: Left gesture be detected 0: No Left gesture be detected
Down	1: Down gesture be detected 0: No Down gesture be detected
Up	1: Up gesture be detected 0: No Up gesture be detected

该寄存器作用于手势识别，实现手势识别输出结果的中断标志。当 BANK0 寄存器 0X41 使能了对应手势中断位后，当 PAJ7620 检测到内置的手势，则对应的寄存器手势标志会置 1，读取标志可清除对应的中断标志位。用户根据读取对应的状态，可知道当前手势识别的结果。

接着我们看下 BANK0 的手势识别中断标志寄存器 2，该地址为 0X44，各位描述如下图所示：

Register Bank 0, ADDR 0x44, Gesture Detection Interrupt Flag		
NAME	Reserved	Wave
BIT #	[7:1]	[0]
ACCESS	R	R
DEFAULT	-	-

NAME	FUNCTION/OPERATION
Wave	1: Wave gesture be detected 0: Wave gesture be detected

该寄存器是承接着 0X43 寄存器，同样也是作用于手势识别，实现手势识别输出结果的中断标志，其中只有 bit0 位有作用，bit1-bit7 为保留位。当 BANK0 寄存器 0X42 使能了“挥动”手势中断位后，PAJ7620 检测到“挥动”的手势，则对应的寄存器手势标志会置 1，读取标志可清除对应的中断标志位。用户根据读取对应的状态，可知道当前手势识别的结果。

接下来我们看下 BANK0 的检测物体亮度寄存器，该地址为 0XB0，寄存器描述如下图所示：

Register Bank 0, ADDR 0xB0, Object Brightness, Report object brightness	
ADDRESS	ADDR 0xB0
NAME	ObjectAvgY[8:1]
BIT #	[7:0]
ACCESS	R
DEFAULT	-

NAME	FUNCTION/OPERATION
ObjectAvgY	Report Object Brightness (Max. value 255).

该寄存器实现获取检测物体亮度值。在接近检测下，当物体在 PAJ7620 的有效检测距离内，读取该寄存器能获得物体的亮度，亮度值为 0~255。

接下来我们看下 BANK0 的检测物体体积大小寄存器，地址为 0XB2 和 0XB1，寄存器描述如下图所示：

Register Bank 0, ADDR 0xB1, ADDR 0xB2, Object Size		
ADDRESS	ADDR 0xB2	ADDR 0xB1
NAME	ObjectSize[11:8]	ObjectSize[7:0]
BIT #	[3:0]	[7:0]
ACCESS	R	R
DEFAULT		

NAME	FUNCTION/OPERATION
ObjectSize	Report Object Size (Max. value 900).

物体的体积大小值由两个寄存器值组合而成，分别是 0XB2 寄存器的低四位值和 0XB1 寄存器八位值。在接近检测下，当物体在 PAJ7620 的有效检测距离内，读取这两个寄存器能获得物体体积大小，体积值为 0~900。

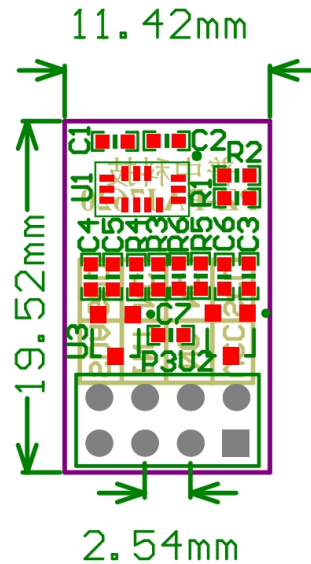
以上就是常用的寄存器，在手册上提供关于 PAJ7620 上电后初始化的数组、手势识别初始化数组，接近检测的初始化数组，以及工作运行的流程，大家可以对其数组的数值和工作流程进行分析。关于更多的寄存器和 PAJ7620 介绍，请看 PAJ7620 芯片手册。

1.2.5 模块使用注意事项

模块属于光学器件，保存时需要注意防尘防潮。在使用时，需保持传感器表面的清洁度，以免导致测量不准。

1.3 结构尺寸

PZ-PAJ7620U 手势识别传感器模块尺寸结构如下图所示：



2 硬件设计

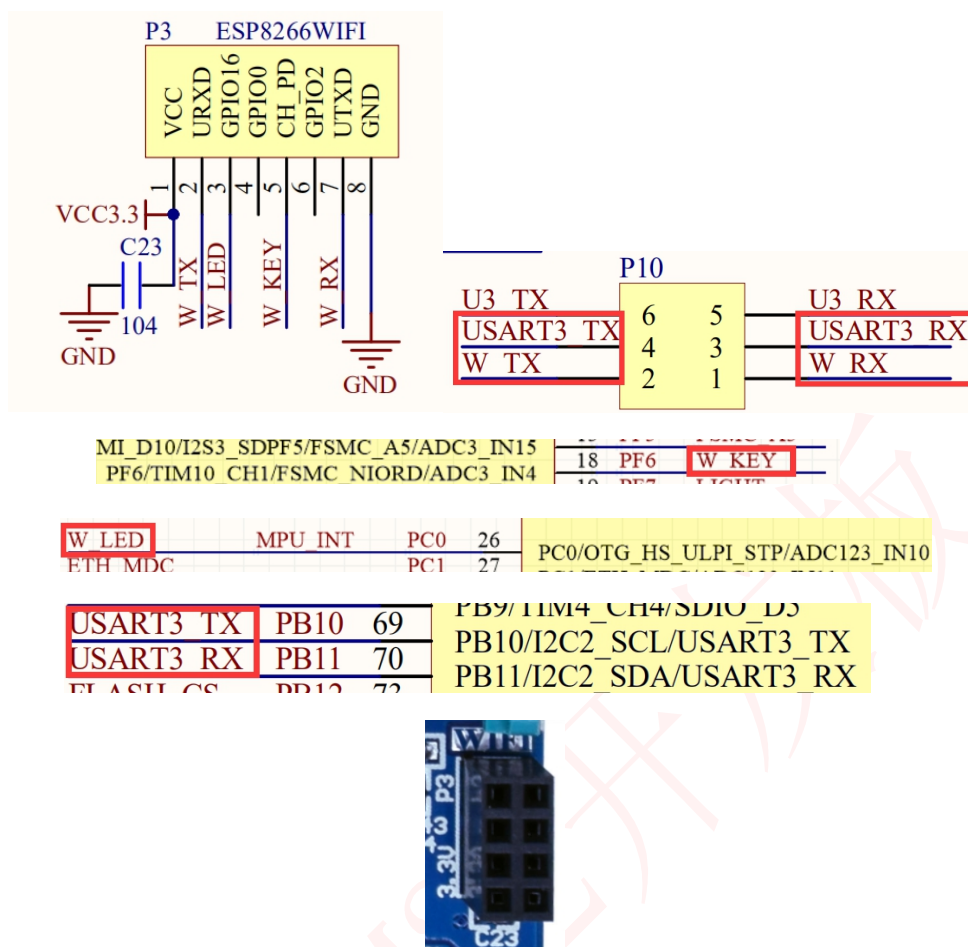
2.1 硬件准备

本实验所需要的硬件资源如下：

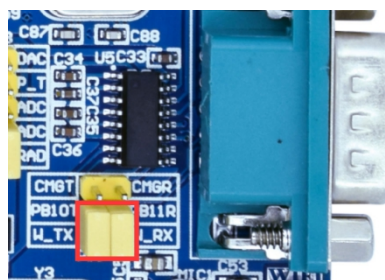
- ①F407 最小系统&天马&麒麟开发板 1 个
- ②TFTLCD 模块（不用彩屏也可用串口输出）
- ③PZ-PAJ7620 模块 1 个
- ④USB 线一条（用于供电和模块与电脑串口调试助手通信）

2.2 模块与开发板连接

PZ-PAJ7620 模块可直接与天马&麒麟开发板板载的 WIFI&蓝牙模块接口(P3) 进行连接，该接口与 MCU 连接原理图（以麒麟为例）如下图所示：



从上图看出,WIFI 接口与 STM32 使用时,必须将 P10 的 USART3_TX(PB10 和 W_TX 以及 USART3_RX (PB11) 和 W_RX 连接,才能完成和 STM32 的连接。默认天马&麒麟的 P10 端子上有黄色跳线帽,直接短接到对应端即可。如下图所示:



对于 F407 最小系统,是没有 WIFI 接口的,所以需要通过杜邦线将模块管脚与上述 STM32 的 I/O 口连接即可。

PZ-PAJ7620 模块与 STM32 的 I/O 口连接关系如下: (PZ-PAJ7620→STM32F407)

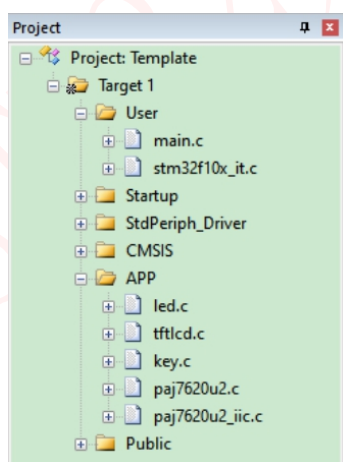
VCC→3.3V
GND→GND

SCL→PB10
SDA→PB11

注意：使用天马&麒麟开发板连接模块时，模块插入 WIFI 接口处请注意方向，可将模块电源与 WIFI 接口丝印电源对应即可。

3 软件设计

本实验在 F407 最小系统&天马&麒麟开发板的 FSMC-TFTLCD 显示实验基础上进行修改，在 APP 文件夹下新建一个 PAJ7620U2 的文件夹。然后新建 paj7620u2.c、paj7620u2.h、paj7620u2_iic.c、paj7620u2_iic.h、paj7620u2_cfg.h 共 5 个文件，paj7620u2.c 文件为手势识别和接近检测的驱动，paj7620u2_iic 文件为底层的 IIC 驱动，而 paj7620u2_cfg.h 文件为存放上电初始化、手势识别初始化、以及接近检测初始化的配置数组。接着在工程目录 APP 文件夹中添加 paj7620u2_iic.c 和 paj7620u2.c 文件，最后添加头文件包含路径。最终的工程如下图所示：



3.1 paj7620u2_cfg.h

本例程代码比较简单，这里我们先说下 paj7620u2_cfg.h 文件，该文件内容是关于初始化配置数组的，代码如下：

```
1. #define INIT_SIZE sizeof(init_Array)/2
2. //上电初始化数组
3. const unsigned char init_Array[][2] = {
4.     {0xEF,0x00},
5.     {0x37,0x07},
```

```

6.      {0x38,0x17},
7.      .....省略部分设置
8.      {0x73,0x35},
9.      {0x74,0x00},
10.     {0x77,0x01},
11. };
12. #define PROXIM_SIZE sizeof(proximity_array)/2
13. //接近检测初始化数组
14. const unsigned char proximity_array[][2]={
15.     {0xEF,0x00},
16.     {0x41,0x00},
17.     {0x42,0x00},
18.     {0x6B,0x50},
19.     .....省略部分设置
20.     {0x6C,0xC3},
21.     {0x6D,0x50},
22.     {0x6E,0xC3},
23.     {0x74,0x05},
24. };
25. #define GESTURE_SIZE sizeof(gesture_array)/2
26. //手势识别初始化数组
27. const unsigned char gesture_array[][2]={
28.     {0xEF,0x00},
29.     {0x74,0x00},
30.     .....省略部分设置
31.     {0xEF,0x00},
32.     {0x41,0xFF},
33.     {0x42,0x01},
34. };

```

以上代码，我们省略了部分（全部贴出来太长了），该文件包含了三个配置数组，分别作用于上电时的初始化、手势识别以及接近检测。该数组为二维数组，每个数组的第一个字节为寄存器号（也就是寄存器地址），第二个字节为要设置的值，比如 {0xEF, 0x00}，就表示在 0xEF 地址，写入 0x00 这个值。

3.2 paj7620u2_iic.c

接下来我们说下 paj7620u2_iic.c 文件，该文件包括 PAJ7620U2 的 IIC 底层驱动读写函数，引脚 IO 初始化以及唤醒命令的函数，关于底层和 IO 初始化就不做讲解了。这里我们讲解下唤醒命令函数，唤醒命令函数代码如下：

```

1. //PAJ7620 唤醒

```



```

2.  void GS_WakeUp(void)
3.  {
4.      GS_IIC_Start();
5.      GS_IIC_Send_Byte(PAJ7620U2_ID); // 发送命令
6.      GS_IIC_Stop(); // 释放总线
7.  }

```

该函数实现触发唤醒，在前面有说到，唤醒需发送唤醒命令，而唤醒命令其实就是写指令，在发送写指令给 PAJ7620U2 后，不必等待 PAJ7620U2 返回的应答，直接释放总线就可以触发一次唤醒。

3.3 paj7620u2.c

接下来我们说下 paj7620u2.c 文件，该文件包括手势识别和接近检测等代码，这里我们讲下：

(1) paj7620u2_selectBank() 函数，具体代码如下：

```

1.  // 选择 PAJ7620U2 BANK 区域
2.  void paj7620u2_selectBank(bank_e bank)
3.  {
4.      switch(bank)
5.      {
6.          case BANK0: GS_Write_Byte(PAJ_REGITER_BANK_SEL, PAJ_BANK0); break; // BANK0 寄存器区域
7.          case BANK1: GS_Write_Byte(PAJ_REGITER_BANK_SEL, PAJ_BANK1); break; // BANK1 寄存器区域
8.      }
9.
10. }

```

该函数实现选择 BANK 区域，通过 bank 的入口参数选择要进入 BANK 的区域。

(2) paj7620u2_wakeup() 函数，具体代码如下：

```

1.  // PAJ7620U2 唤醒
2.  u8 paj7620u2_wakeup(void)
3.  {
4.      u8 data=0x0a;
5.      GS_WakeUp(); // 唤醒 PAJ7620U2
6.      delay_ms(5); // 唤醒时间>400us
7.      GS_WakeUp(); // 唤醒 PAJ7620U2
8.      delay_ms(5); // 唤醒时间>400us
9.      paj7620u2_selectBank(BANK0); // 进入 BANK0 寄存器区域
10.     data = GS_Read_Byte(0x00); // 读取状态
11.     if(data!=0x20) return 0; // 唤醒失败

```

```
12.
13.     return 1;
14. }
```

该函数实现对 PAJ7620U2 唤醒，在代码中我们发送两次 GS_Wake() 唤醒命令，因为发送一次，可能会遇到唤醒不起 PAJ7620U2。在发送唤醒命令后需等待大于 700us 的时间，然后切换到 BANK0 区域，读取 0x00 寄存器，判断读取的值是否为 0x20，若是则唤醒成功，否则唤醒失败。

(3) paj7620u2_init() 函数，具体代码如下：

```
1. //PAJ7620U2 初始化
2. //返回值: 0:失败 1:成功
3. u8 paj7620u2_init(void)
4. {
5.     u8 i;
6.     u8 status;
7.
8.     GS_i2c_init();//IIC 初始化
9.     status = paj7620u2_wakeup();//唤醒 PAJ7620U2
10.    if(!status) return 0;
11.    paj7620u2_selectBank(BANK0);//进入 BANK0 寄存器区域
12.    for(i=0;i<INIT_SIZE;i++)
13.    {
14.        GS_Write_Byte(init_Array[i][0],init_Array[i][1]);//初始化 PAJ7620U2
15.    }
16.    paj7620u2_selectBank(BANK0);//切换回 BANK0 寄存器区域
17.
18.    return 1;
19. }
```

该函数实现对 PAJ7620U2 初始化，先对 IIC 通信的 IO 引脚初始化，然后调用 paj7620u2_wakeup() 函数对 PAJ7620U2 进行唤醒，唤醒成功后，由于后面要调用 init_Array 初始化数组，初始化时是从 BANK0 区域开始的，所以这里先选择进入 BANK0 区域，初始化数组配置完毕后，又切换回 BANK0 区域，这时 PAJ7620U2 初始化完成。

(4) Gesrure_test() 函数，具体代码如下：

```
1. //手势识别测试
2. void Gesture_test(void)
3. {
4.     u8 i;
5.     u8 status;
6.     u8 key;
```

```

7.      u8 data[2]={0x00};
8.      u16 gesture_data;
9.      u8 ledflash=0;
10.
11.      paj7620u2_selectBank(BANK0);//进入 BANK0 寄存器区域
12.      for(i=0;i<GESTURE_SIZE;i++)
13.      {
14.          GS_Write_Byte(gesture_arry[i][0],gesture_arry[i][1]);//手势识别模式初始化
15.      }
16.      paj7620u2_selectBank(BANK0);//切换回 BANK0 寄存器区域
17.      i=0;
18.      FRONT_COLOR=BLUE;//设置字体为蓝色
19.      LCD_Fill(30,170,300,300,WHITE);
20.      LCD_ShowString(30,180,200,16,16,"KEY_UP: Exit the test");
21.      LCD_ShowString(30,210,200,16,16,"Gesture test");
22.      printf("KEY_UP: Exit the test\r\n");
23.      printf("Gesture test\r\n");
24.      FRONT_COLOR=RED;//设置字体为蓝色
25.      while(1)
26.      {
27.          key = KEY_Scan(0);
28.          if(key==KEY_UP_PRESS)
29.          {
30.              GS_Write_Byte(PAJ_SET_INT_FLAG1,0X00);//关闭手势识别中断输出
31.              GS_Write_Byte(PAJ_SET_INT_FLAG2,0X00);
32.              break;
33.          }
34.          status = GS_Read_nByte(PAJ_GET_INT_FLAG1,2,&data[0]);//读取手势状态
35.          if(!status)
36.          {
37.              gesture_data =(u16)data[1]<<8 | data[0];
38.              if(gesture_data)
39.              {
40.                  switch(gesture_data)
41.                  {
42.                      case GES_UP:    LCD_ShowString(110,250,200,16,24,"UP          ");
43.                                  printf("Up\r\n"); ledflash=1; break; //向上
44.                      case GES_DOWM: LCD_ShowString(100,250,200,16,24,"Down          ");
45.                                  printf("Down\r\n"); ledflash=1;break; //向下
46.                      case GES_LEFT:  LCD_ShowString(100,250,200,16,24,"Left          ");
47.                                  printf("Left\r\n");ledflash=1; break; //向左
48.                      case GES_RIGHT: LCD_ShowString(100,250,200,16,24,"Right         ");
49.                                  printf("Right\r\n"); ledflash=1; break; //向右
50.                      case GES_FORWARD: LCD_ShowString(80,250,200,16,24,"Forward        ");

```

```

51.                                     printf("Forward\r\n"); ledflash=1; break; //向前
52.         case GES_BACKWARD: LCD_ShowString(80,250,200,16,24,"Backward   ");
53.                                     printf("Backward\r\n"); ledflash=1; break; //向后
54.         case GES_CLOCKWISE: LCD_ShowString(70,250,200,16,24,"Clockwise  ");
55.                                     printf("Clockwise\r\n");ledflash=1;break; //顺时针
56.         case GES_COUNT_CLOCKWISE: LCD_ShowString(50,250,200,16,24,"AntiClockwi
se");
57.                                     printf("AntiClockwise\r\n"); ledflash=1; break; //逆时针
58.         case GES_WAVE: LCD_ShowString(100,250,200,16,24,"Wave       ");
59.                                     printf("Wave\r\n"); ledflash=1; break; //挥动
60.         default: ledflash=0; break;
61.
62.     }
63.     if(ledflash)//DS1 闪烁
64.     {
65.         LED2=0;delay_ms(80);LED2=1;delay_ms(80);
66.         LED2=0;delay_ms(80);LED2=1;delay_ms(80);
67.         delay_ms(300);
68.         LCD_ShowString(40,250,200,16,24,"
");
69.         ledflash=0;
70.     }
71. }
72.
73. }
74. delay_ms(50);
75. i++;
76. if(i==5)
77. {
78.     LED2=!LED2;//提示系统正在运行
79.     i=0;
80. }
81. }
82. }

```

该函数实现手势识别功能，一开始先对 PAJ7620U2 手势识别检测进行初始化，配置其 gesture_array 数组的参数值。这里说一下，该数组有使能 9 个手势识别的中断标志输出的配置，初始化完毕后，在 while 循环中一直读取手势中断标志寄存器，当识别到相应的手势，手势标志会置 1，读取寄存器标志会自动清 0。根据手势的标志，在 LCD 屏幕上显示识别到的手势结果，DS1 灯会闪烁，同时手势结果打印到串口上，DS0 灯闪烁表示检测正在运行。当按下 KEY_UP 按键，关闭 9 个手势识别的检测输出，退出测试。

(5) Ps_test()函数，具体代码如下：

```
1. //接近检测测试
2. void Ps_test(void)
3. {
4.     u8 i;
5.     u8 key;
6.     u8 data[2]={0x00};
7.     u8 obj_brightness=0;
8.     u16 obj_size=0;
9.
10.    paj7620u2_selectBank(BANK0);//进入 BANK0 寄存器区域
11.    for(i=0;i<PROXIM_SIZE;i++)
12.    {
13.        GS_Write_Byte(proximity_array[i][0],proximity_array[i][1]);//接近检测模式初始化
14.    }
15.    paj7620u2_selectBank(BANK0);//切换回 BANK0 寄存器区域
16.    i=0;
17.    FRONT_COLOR=BLUE;//设置字体为蓝色
18.    LCD_Fill(30,170,300,300,WHITE);
19.    LCD_ShowString(30,180,200,16,16,"KEY_UP: Exit the test");
20.    LCD_ShowString(30,210,200,16,16,"Ps test");
21.    LCD_ShowString(30,240,200,16,16,"Brightness");
22.    LCD_ShowString(160,240,200,16,16,"Size");
23.    printf("KEY_UP: Exit the test\r\n");
24.    printf("Ps test\r\n");
25.    FRONT_COLOR=RED;//设置字体为蓝色
26.
27.    while(1)
28.    {
29.        key = KEY_Scan(0);
30.        if(key==KEY_UP_PRESS) break;
31.
32.        obj_brightness = GS_Read_Byte(PAJ_GET_OBJECT_BRIGHTNESS);//读取物体亮度
33.        data[0] = GS_Read_Byte(PAJ_GET_OBJECT_SIZE_1);//读取物体大小
34.        data[1] = GS_Read_Byte(PAJ_GET_OBJECT_SIZE_2);
35.        obj_size = ((u16)data[1] & 0x0f)<<8 | data[0];
36.        LCD_ShowxNum(50,270,obj_brightness,3,24,0);
37.        LCD_ShowxNum(152,270,obj_size,3,24,0);
38.        printf("obj_brightness: %d\r\n",obj_brightness);
39.        printf("obj_size: %d\r\n",obj_size);
40.
41.        delay_ms(100);
42.        i++;
43.        if(i==5)
```

```

44.     {
45.         LED2=!LED2;//提示系统正在运行
46.         i=0;
47.     }
48. }
49.
50. }

```

该函数实现接近检测，获取接近物体的体积大小和亮度的数据。同样开始对接近检测进行初始化，配置其 proximity_array 的数组，初始化完成后，在 while(1) 循环中，读取获取体积大小和亮度的寄存器，读到的数据显示在 LCD 屏幕上，同时打印到串口上，DS0 闪烁表示检测正在运行，当按下 KEY_UP 按键，退出测试。

3.4 main.c

最后我们说下 main.c，代码如下：

```

1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "paj7620u2.h"
8.
9.
10. /*****
11.  * 函数名      : main
12.  * 函数功能    : 主函数
13.  * 输 入      : 无
14.  * 输 出      : 无
15.  *****/
16. int main()
17. {
18.     SysTick_Init(168);
19.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
20.     LED_Init();
21.     USART1_Init(115200);
22.     TFTLCD_Init(); //LCD 初始化
23.     KEY_Init(); //按键初始化
24.
25.

```

```

26.     FRONT_COLOR=RED;
27.     LCD_ShowString(30,10,tftlcd_data.width,tftlcd_data.height,16,"PRECHIN-STM32");
28.     LCD_ShowString(30,30,tftlcd_data.width,tftlcd_data.height,16,"Sensor PAJ7620U2 TEST");

29.     LCD_ShowString(30,50,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.net");
30.     printf("Sensor PAJ7620U2 TEST\r\n");
31.     FRONT_COLOR=BLUE;
32.     while(!paj7620u2_init())//PAJ7620U2 传感器初始化
33.     {
34.         printf("PAJ7620U2 Error!!!\r\n");
35.         LCD_ShowString(30,140,200,16,16,"PAJ7620U2 Error!!!");
36.         delay_ms(500);
37.         LCD_ShowString(30,140,200,16,16,"");
38.         delay_ms(500);
39.         LED1=!LED1;//DS0 闪烁
40.
41.     }
42.     printf("PAJ7620U2 OK\r\n");
43.     LCD_ShowString(30,140,200,16,16,"PAJ7620U2 OK");
44.     while(1)
45.     {
46.         paj7620u2_sensor_test();//PAJ7620U2 传感器测试
47.     }
48. }

```

此部分代码比较简单，对用到的外设进行初始化，然后通过调用 `paj7620u2_sensor_test()` 函数，进入 PZ-PAJ7620 模块的主测试程序，对 PZ-PAJ7620 的手势识别和接近检测功能进行测试。

至此，软件实现部分介绍完了，我们接下来看代码验证。

4 实验现象

首先，请先确保硬件都已经连接好：

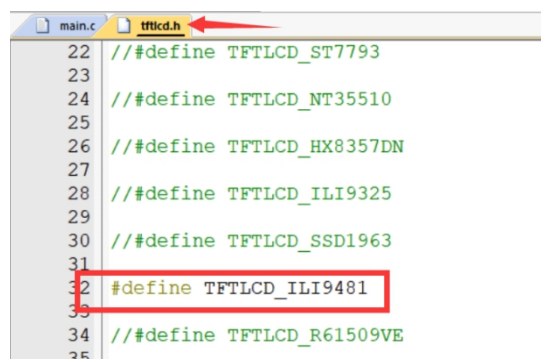
①PZ-PAJ7620 模块与 F407 最小系统&天马&麒麟开发板连接（可参考硬件设计小节）

②开发板插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口输出。

③使用 USB 线给开发板供电。

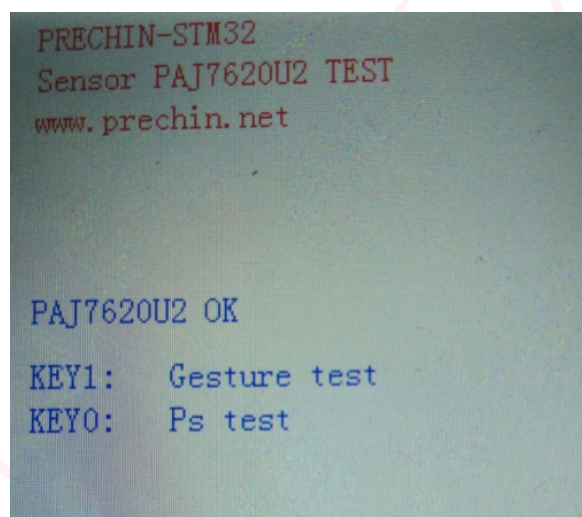
注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱

动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



```
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）LCD 界面显示如下图所示：



```
PRECHIN-STM32
Sensor PAJ7620U2 TEST
www.prechin.net

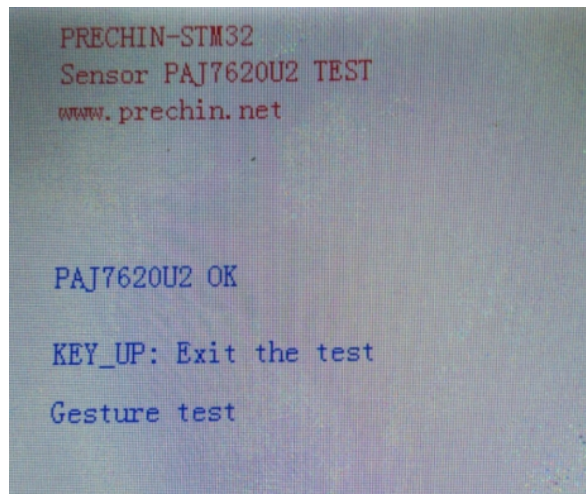
PAJ7620U2 OK

KEY1:  Gesture test
KEY0:  Ps test
```

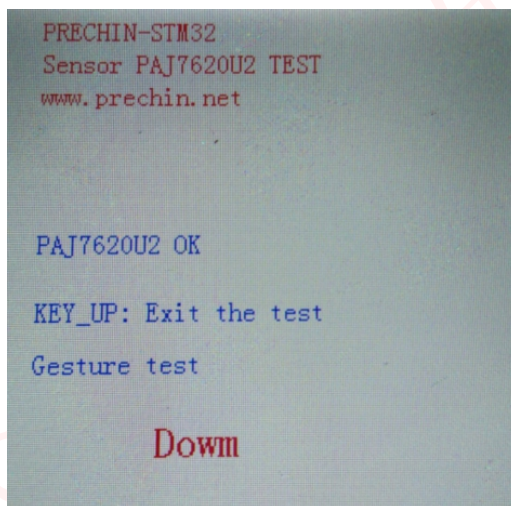
可以看到，LCD 屏幕显示 PAJ7620U2 OK，表示 PAJ7620U2 传感器初始化通过，同时显示了 KEY1/KEY0 测试功能选项，KEY1 手势识别测试、KEY0 接近检测测试。

4.1 手势识别测试

在主菜单界面，按 KEY1，则可进入此项测试，此项测试为手势识别测试，主界面如下图所示：



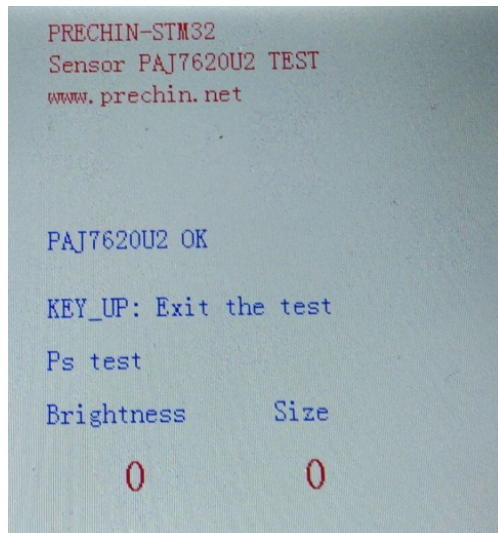
当手在 PZ-PAJ7620 模块上方手势滑动，会识别出 9 个的手势识别，识别结果显示在 LCD 屏幕上，同时 DS1 闪烁。如下图所示为“下”的识别效果。



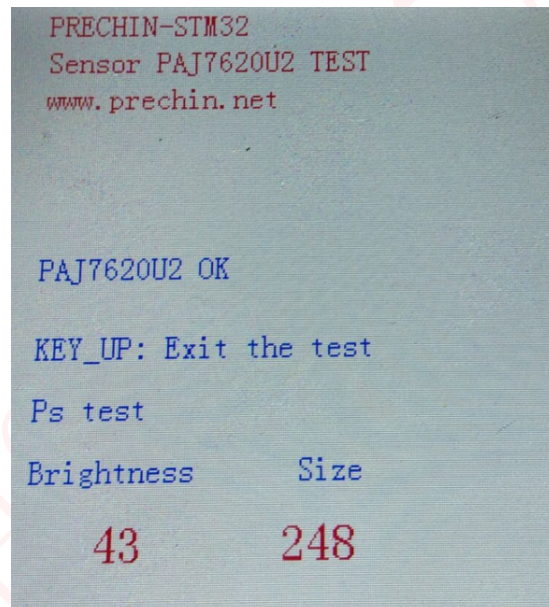
按下 KEY_UP 按键可退出手势识别测试，返回主菜单页面。

4.2 接近检测测试

在主菜单页面，按 KEY0，可进入此项测试，此项测试接近检测功能。接近检测测试主界面如下图所示：



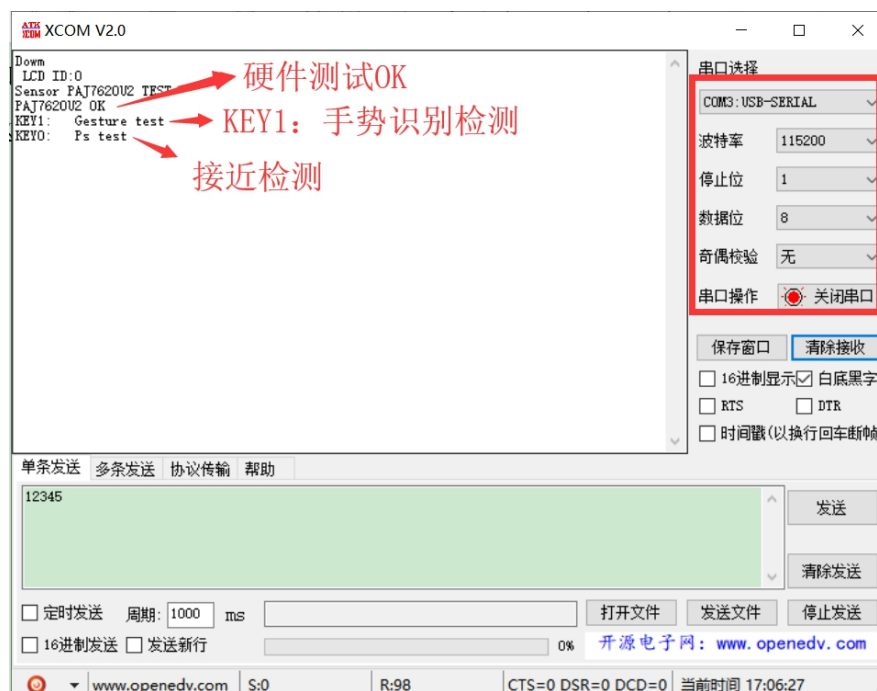
当手在模块上方动作时，模块检测到物体的体积大小和亮度数据也在变化如下图所示：



最后，按下 KEY_UP 按键可退出接近检测测试，返回主菜单页面。

4.3 串口输出

对于没有 TFTLCD 彩屏的用户，我们使用 PZ-PAJ7620 模块的时候，也可以使用串口来输出信息。按照串口输出提示信息进行操作，与上述操作是一样的，只不过这里使用串口输出显示。串口输出如下图所示：



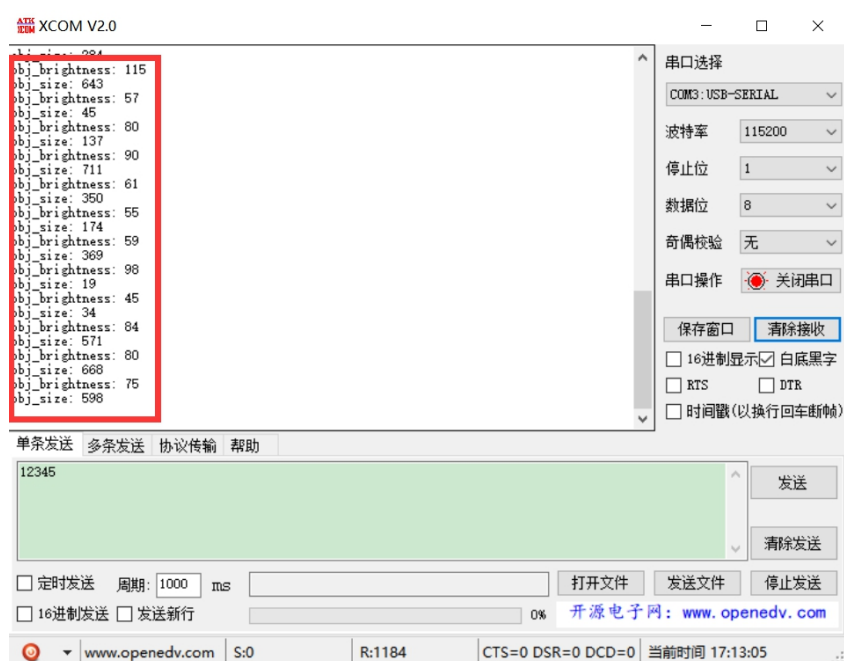
可以看到，串口助手显示 PAJ7620U2 OK，表示 PAJ7620U2 传感器初始化通过，同时显示了 KEY1/KEY0 测试功能选项，KEY1 手势识别测试、KEY0 接近检测测试。

按 KEY1，则可进入手势识别测试，当手在 PZ-PAJ7620 模块上方手势滑动，会识别出 9 个的手势识别，识别结果显示在串口助手上，同时 DS1 闪烁。如下图所示为“左”“右”的识别效果。



按下 KEY_UP 按键可退出手势识别测试，返回主菜单页面。

在主菜单页面，按 KEY0，可进入接近检测功能。当手在模块上方动作时，模块检测到物体的体积大小和亮度数据也在变化如下图所示：



最后，按下 KEY_UP 按键可退出接近检测测试，返回主菜单页面。

至此，关于 PZ-PAJ7620 模块的使用介绍，我们就讲完了，本手册介绍 PZ-PAJ7620U2 模块的使用，有助于大家快速学会 PZ-PAJ7620 模块的使用。

5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）